Department of Computer Science and Engineering

CSA06 – Design and Analysis of Algorithms

CO3 – Case Study: Divide-and-Conquer Engine of "KartIQ", a Retail Data Analytics Platform

Item	Details
Student Name	Deepak S
Student Reg. No.	192511124
Course Outcome Assessed	CO3 – Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Mode / Duration	Individual / 1 Week

(a) Understanding the Case

KartIQ handles about 10,000,000 orders per day and ranks 100,000 sellers. The table identifies, for each module, the computational requirement, the key issues and the consequence of a poor algorithm choice.

Module	Computational requirement	Key issues	Consequence of a poor choice
1 – Nightly Order Consolidation	Sort 10^7 orders by timestamp for the audit report.	**Stability:** equal timestamps must keep gateway arrival order. **Memory:** 10^7 records may not fit in RAM. **Worst case:** must be guaranteed, since the job has a deadline.	An unstable sort (e.g. plain Quick Sort or Heap Sort) breaks gateway-wise reconciliation; an $O(n^2)$ sort misses the nightly deadline (10^{14} steps).
2 – Live Seller Leaderboard	Re-sort 100,000 scores every few seconds.	**Nearly sorted input** between refreshes. **Worst-case behaviour:** first-element pivot on sorted data gives unbalanced partitions. **Stack depth** can reach n.	$\Theta(n^2) \approx 5\times10^9$ comparisons per refresh and O(n) recursion depth: the leaderboard freezes during sales (current symptom).
3 – Order Lookup	Find an order by timestamp; return the first of several with the same timestamp.	**Sorted-data precondition:** depends on Module 1 output. **Duplicates:** an ordinary binary search may return any match.	Linear scan costs up to 10^7 comparisons (seconds); a plain binary search returns the wrong duplicate; on unsorted data results are wrong.
4 – Recommendation Scores	C = A × B for n × n matrices, n = 1,024.	**Matrix size:** $n^3 \approx 1.07\times10^9$ scalar multiplications. **Memory and overheads** of recursive methods; n should be a power of 2.	The conventional method makes the refresh slow (once a day); a naive Strassen to 1×1 gives little real gain and uses extra memory.

(b) Module 1 – Merge Sort for Nightly Order Consolidation

1. Divide, Conquer and Combine

- **Divide:** split the array of n orders into two halves of about n/2 orders each (at the middle index).
- **Conquer:** recursively sort each half; an array of size 1 is already sorted (base case).
- **Combine:** merge the two sorted halves with two pointers, always copying the smaller front element into a temporary array. If the timestamps are equal, the element from the **left** half is copied first. This rule makes the sort stable.

2. Trace on the sample orders

Input: (105,O1), (102,O2), (105,O3), (101,O4), (102,O5), (103,O6). Division: [O1 O2 O3] | [O4 O5 O6], then [O1] | [O2 O3], [O2] | [O3], [O4] | [O5 O6], [O5] | [O6]. Every merge is shown below.

Merge	Left list	Right list	Comparisons made	Result	No. of comparisons
M1	[O2(102)]	[O3(105)]	102 vs 105 → O2	[O2(102), O3(105)]	1
M2	[O1(105)]	[O2(102), O3(105)]	105 vs 102 → O2; 105 vs 105 (equal) → O1 (left first); left empty, copy O3	[O2(102), O1(105), O3(105)]	2
M3	[O5(102)]	[O6(103)]	102 vs 103 → O5	[O5(102), O6(103)]	1
M4	[O4(101)]	[O5(102), O6(103)]	101 vs 102 → O4; left empty, copy O5, O6	[O4(101), O5(102), O6(103)]	1
M5 (final)	[O2(102), O1(105), O3(105)]	[O4(101), O5(102), O6(103)]	102 vs 101 → O4; 102 vs 102 (equal) → O2 (left first); 105 vs 102 → O5; 105 vs 103 → O6; right empty, copy O1, O3	[O4(101), O2(102), O5(102), O6(103), O1(105), O3(105)]	4

Sorted output: (101,O4), (102,O2), (102,O5), (103,O6), (105,O1), (105,O3). Total comparisons = 1 + 2 + 1 + 1 + 4 = **9**.

3. Stability demonstration

In the input O2 arrives before O5 (both 102) and O1 arrives before O3 (both 105). In the output O2 still precedes O5 and O1 still precedes O3. The reason is the merge rule: when keys are equal, the element from the left sublist is taken first, and the left sublist always holds elements that came earlier in the original array. This was applied in M2 (O1 before O3) and in M5 (O2 before O5). Hence orders with the same second keep their gateway arrival order, which is what the auditors need.

4. Recurrence and solution

```
T(n) = 2T(n/2) + (n − 1),   T(1) = 0
```

Each level of the recursion tree does at most n comparisons in total (the merges at one level together touch all n elements) and there are $\log_2 n$ levels. Expanding for n = 2^k:

```
T(n) = 2T(n/2) + n − 1 = 4T(n/4) + 2n − 3 = … = 2^k·T(1) + k·n − (2^k − 1)
T(n) = n·log₂ n − n + 1  = Θ(n log n)
```

By the Master Theorem: a = 2, b = 2, f(n) = Θ(n) = Θ(n^(log_b a)), so Case 2 gives Θ(n log n). The bound holds for best, average and worst cases, so the nightly deadline is predictable.

5. Estimate for 10,000,000 orders

- $\log_2(10^7) \approx 23.25$, so $n \cdot \log_2 n - n + 1 \approx 232.5\times10^6 - 10\times10^6 \approx$ **2.23×10^8 comparisons**.
- Exact worst case for non-power-of-2 n: n log₂ n − 2^ log₂ n + 1 = $10^7\times24 - 16{,}777{,}216 + 1 = 223{,}222{,}785 \approx 2.23\times10^8$.
- For comparison, an $O(n^2)$ sort needs about $n^2/2 = 5\times10^{13}$ comparisons, roughly 220,000 times more.

6. Memory requirement and handling data larger than RAM

- Merge Sort needs an auxiliary array of size n: O(n) extra space. Assuming 16 bytes per record (8-byte timestamp + 8-byte order ID), the data is ≈160 MB and the temporary array another ≈160 MB. This can be reduced by allocating one buffer once and alternating source and destination between levels (no repeated allocation).

- **If the data does not fit in RAM – external merge sort:** (1) read chunks of the orders in arrival order, sized to fit in memory (e.g. 1,000,000 records → 10 runs), sort each chunk with in-memory Merge Sort and write it to disk as a sorted run; (2) perform a k-way merge of the runs using a min-heap that holds one record per run, comparing by (timestamp, run number). Because runs are made from consecutive arrival chunks, using the run number as the tie-breaker keeps the merge stable. With k runs, each record is compared about $\log_2 k$ times and the data is read and written only about twice (sequential disk I/O). If there are too many runs, merge in multiple passes: passes = log_k(number of runs) .

(c) Module 2 – Quick Sort for the Live Leaderboard

1. One partition step on the sample scores

Scores: 42, 17, 63, 8, 55, 29. Pivot = first element = 42. Pointer i scans from the left for an element > pivot; pointer j scans from the right for an element ≤ pivot.

Step	Action	Array after step
1	i stops at 63 (index 2, > 42); j stops at 29 (index 5, ≤ 42); i < j → swap 63 and 29	42, 17, 29, 8, 55, 63
2	i stops at 55 (index 4); j moves left past 63, 55 and stops at 8 (index 3); i > j → stop	42, 17, 29, 8, 55, 63
3	Swap pivot with a[j]: swap 42 and 8	8, 17, 29, **42**, 55, 63

The pivot 42 is now in its final position (index 3). Left part [8, 17, 29] has elements ≤ 42 and right part [55, 63] has elements > 42; both are then sorted recursively.

2. Why the first-element pivot makes the leaderboard slow

Between refreshes the list is almost sorted. The first element is then the smallest (or nearly the smallest) value, so the partition puts the pivot at the beginning: one part has 0 (or very few) elements and the other has n − 1. The problem size shrinks by only one at each step, giving n levels of recursion with ~n comparisons each. Sorted data is the worst case for this pivot rule. In addition, the recursion depth becomes n (= 100,000), which risks a stack overflow and adds call overhead.

3. Recurrences and solutions

Best case (pivot is the median, balanced split):

$$T(n) = 2T(n/2) + n,\quad T(1) = 0 \quad \rightarrow \quad T(n) = n \log_2 n = \Theta(n \log n)$$

(Master Theorem, Case 2: a = 2, b = 2, f(n) = Θ(n).)

Worst case (pivot is min or max, split 0 : n − 1):

$$T(n) = T(n-1) + (n-1),\quad T(1) = 0$$
$$T(n) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = \Theta(n^2)$$

4. Estimate for 100,000 nearly sorted scores

Case	Formula	Comparisons for n = 100,000
Current (first pivot, nearly sorted → worst case)	n(n−1)/2	100,000 × 99,999 / 2 ≈ **$4.99995\times10^9 \approx 5\times10^9$**
Balanced pivot (best case)	$n\cdot\log_2 n$, $\log_2 10^5 \approx 16.61$	≈ **1.66×10^6**
Randomised / median-of-three (expected)	≈ 2n ln n	≈ $2 \times 10^5 \times 11.51 \approx$ **2.3×10^6**

A good pivot is therefore about 2,000–3,000 times fewer comparisons than the current implementation.

5. Improved pivot strategy

- **Median-of-three:** choose the median of the first, middle and last elements as the pivot (swap it to the front before partitioning). For sorted or nearly sorted data the middle element is close to the true median, so the split is almost balanced and the running time is ≈ n log n.
- **Randomised pivot:** pick a random index. No fixed input (including sorted input) can force the worst case; the expected time is ≈ 1.39 n $\log_2$ n and the probability of the $\Theta(n^2)$ case is negligible.
- **Supporting changes:** recurse on the smaller partition and loop on the larger one (stack depth O(log n)); use Insertion Sort for partitions below ~16 elements.

Justification: the cause of the slowdown is the predictability of the pivot on sorted data. Median-of-three removes this for the sorted pattern at almost no cost (three comparisons), and a random pivot removes it for every input pattern. Both keep Quick Sort in place, with O(log n) extra stack.

(d) Module 3 – Binary Search for Order Lookup

1. Why the sorted order from Module 1 is essential

Binary search compares the key with the middle element and discards half of the array. This is correct only if all elements on one side of the middle are ≤ it and all on the other side are ≥ it, i.e. the list is sorted. On unsorted data, discarding half could discard the target, giving wrong "not found" results. Sorting once (Θ(n log n), nightly) allows each of the many lookups to cost only Θ(log n). Module 1 also keeps equal timestamps in arrival order, so the first matching order is also the earliest arrived one.

2. Recurrence and solution

```
T(n) = T(n/2) + 1,  T(1) = 1
T(n) = T(n/4) + 2 = … = T(1) + log₂ n   →   T(n) =  log₂ n  + 1 = Θ(log n)
```

(Master Theorem: a = 1, b = 2, f(n) = $\Theta(1) = \Theta(n^0)$, Case 2 → Θ(log n).)

3. Maximum comparisons for 10,000,000 orders

$\log_2$ 10,000,000 + 1 = 23.25 + 1 = **24 comparisons** at most (each step halves the range: 10^7 → 5×10^6 → … → 1 in about 24 steps), compared with up to 10,000,000 comparisons for a linear scan.

4. Modified search – first order with a given timestamp

When a match is found at mid, do not stop. Record it as a candidate and continue searching the left half for an earlier occurrence.

```
FirstOccurrence(A[0..n-1], key):
    lo ← 0;  hi ← n − 1;  ans ← −1
    while lo ≤ hi:
        mid ← lo + (hi − lo) / 2
        if A[mid].timestamp = key:   ans ← mid;  hi ← mid − 1   // keep looking left
        else if A[mid].timestamp < key:   lo ← mid + 1
        else:   hi ← mid − 1
    return ans        // −1 if the timestamp does not exist
```

Trace on the Module 1 output, timestamps [101, 102, 102, 103, 105, 105], key = 102: lo=0, hi=5, mid=2 (102) → ans=2, hi=1; mid=0 (101 < 102) → lo=1; mid=1 (102) → ans=1, hi=0; loop ends. Returns index 1, i.e. order **O2**, the first order with timestamp 102. The worst case remains $\log_2$ n + 1 comparisons, because the search always continues until the range is empty.

(e) Module 4 – Strassen's Method for Recommendation Scores

1. Strassen's seven products and reconstruction

Partition A and B into four n/2 × n/2 blocks (for 2 × 2 matrices the blocks are scalars): A = [a11 a12; a21 a22], B = [b11 b12; b21 b22].

```
M1 = (a11 + a22)(b11 + b22)
M2 = (a21 + a22) b11
M3 = a11 (b12 − b22)
M4 = a22 (b21 − b11)
M5 = (a11 + a12) b22
M6 = (a21 − a11)(b11 + b12)
M7 = (a12 − a22)(b21 + b22)

C11 = M1 + M4 − M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 − M2 + M3 + M6
```

2. Verification on the sample matrices

A = [[1, 7], [3, 5]], B = [[1, 3], [7, 5]]: a11=1, a12=7, a21=3, a22=5; b11=1, b12=3, b21=7, b22=5.

Product	Calculation	Value
M1	(1 + 5)(1 + 5) = 6 × 6	36
M2	(3 + 5) × 1 = 8 × 1	8
M3	1 × (3 − 5) = 1 × (−2)	−2
M4	5 × (7 − 1) = 5 × 6	30
M5	(1 + 7) × 5 = 8 × 5	40
M6	(3 − 1)(1 + 3) = 2 × 4	8
M7	(7 − 5)(7 + 5) = 2 × 12	24

Element	Strassen	Conventional (check)	Match
C11	M1 + M4 − M5 + M7 = 36 + 30 − 40 + 24 = 50	1×1 + 7×7 = 50	
C12	M3 + M5 = −2 + 40 = 38	1×3 + 7×5 = 38	
C21	M2 + M4 = 8 + 30 = 38	3×1 + 5×7 = 38	
C22	M1 − M2 + M3 + M6 = 36 − 8 − 2 + 8 = 34	3×3 + 5×5 = 34	

Result: C = [[50, 38], [38, 34]]. Strassen used 7 multiplications instead of 8.

3. Recurrence and solution

```
T(n) = 7T(n/2) + Θ(n²),  T(1) = 1
Master Theorem: a = 7, b = 2, n^(log₂7) = n^2.807 dominates n²  →  T(n) = Θ(n^2.807)
```

Counting only multiplications, M(n) = 7M(n/2), M(1) = 1, so M(2^k) = 7^k.

4. Comparison for n = 1,024 (= 2^{10})

Method	Scalar multiplications	Remarks
Conventional	$n^3 = 1{,}024^3$ = **1,073,741,824** (≈1.07×10^9)	—
Strassen (recursing to 1×1)	7^{10} = **282,475,249** (≈2.82×10^8)	Ratio 0.263: 3.80× fewer multiplications, a 73.7% saving

Strassen, total operations (mult. + add.)	Additions $A(2^k) = 6(7^k - 4^k) = 1{,}688{,}560{,}038$ → total ≈ 1.97×10^9	Conventional total = 1,073,741,824 + 1,072,693,248 ≈ 2.15×10^9; gain only ≈ 8%

5. Why Strassen may still not be faster in practice

- **Extra additions:** 18 matrix additions/subtractions per level. When all operations are counted, the saving falls to about 8% for full recursion.
- **Recursion overhead and memory:** temporary matrices for 10 sums and 7 products at each level (extra $O(n^2)$ memory) and many small function calls.
- **Cache behaviour:** the conventional blocked algorithm and optimised libraries (BLAS) exploit cache and SIMD far better than the recursive splitting at small sizes.
- **Other limits:** requires n to be a power of 2 (padding otherwise) and is numerically less stable for floating-point scores.

6. Practical hybrid

Use Strassen only while the blocks are large and switch to the conventional (cache-blocked) multiplication below a cut-off size, e.g. $n \le 64$ (best value found by benchmarking, typically 64–128). For n = 1,024: 4 Strassen levels (1,024 → 512 → 256 → 128 → 64), giving $7^4 = 2{,}401$ conventional 64 × 64 products of $64^3 = 262{,}144$ multiplications each:

```
Multiplications = 2,401 × 262,144 = 629,407,744  (≈ 41% fewer than 1.07×10⁹)
Total operations ≈ 6.29×10⁸ + 6.72×10⁸ ≈ 1.30×10⁹  vs  2.15×10⁹ conventional (≈ 39% fewer)
```

The hybrid keeps most of Strassen's saving, avoids the deep-recursion overhead, limits temporary memory, and the base case runs on cache-friendly code. Since n = 1,024 is already a power of 2, no padding is needed.

(f) Final Solution – Design and Justification

Module	Chosen algorithm	Time complexity	Extra memory	Reason for the choice	Main trade-off	Change the decision when…
1 – Order consolidation	Stable Merge Sort (external merge sort if data > RAM)	$\Theta(n \log n)$ in all cases (≈2.2×10^8 comparisons for 10^7)	$O(n)$ (≈160 MB); $O(\text{run size})$ for external sort	Stable (keeps gateway order) and guaranteed worst case for the nightly deadline.	Uses $O(n)$ extra memory to get stability + guaranteed time.	Memory is very tight and stability is not required → in-place Heap Sort; or data far exceeds RAM → external merge sort.
2 – Leaderboard	Quick Sort with median-of-three / random pivot, Insertion Sort for small parts	Expected $\Theta(n \log n)$ (≈2.3×10^6 for 10^5); worst case $\Theta(n^2)$ is very unlikely	$O(\log n)$ stack	Removes the sorted-data worst case; in place and fast in practice.	No hard worst-case guarantee and it is not stable.	Only a handful of scores change per refresh → Insertion Sort/Timsort (≈ $O(n)$); an adversarial input is a concern → Introsort (switch to Heap Sort at depth $2 \log n$).

3 – Order lookup	Binary Search, first-occurrence version	Θ(log n) (≤ 24 comparisons for 10^7)	O(1)	Very fast lookups on the sorted Module 1 list; returns the first matching order.	Needs sorted data; new orders cannot be added without re-sorting/merging.	Orders arrive continuously (frequent updates) → balanced tree/B-tree index; lookups are by order ID → hash index (O(1)).
4 – Matrix scores	Hybrid Strassen (cut-off ≈ 64) with conventional blocked multiplication below it	Θ(n^2.81) above the cut-off; ≈6.3×10^8 mult. for n = 1,024 vs 1.07×10^9	O(n^2) temporary matrices	Fewer multiplications than n^3 while avoiding Strassen's small-size overhead.	Speed gain vs extra memory, complexity and slightly lower numerical stability.	Matrices are sparse, small, or a GPU/BLAS library is available → use the optimised conventional/sparse routine; strict accuracy needed → conventional method.

Overall: the redesigned platform sorts the nightly orders in a predictable Θ(n log n) time while preserving gateway order (Merge Sort), refreshes the leaderboard in about 10^6 comparisons instead of 5×10^9 (improved Quick Sort), answers lookups in at most 24 comparisons (Binary Search on the Module 1 output) and cuts the recommendation computation by about 40% (hybrid Strassen). The four modules are linked: Module 3 relies on the sorted, stable output of Module 1.

Code of Conduct

I, Deepak S (Reg. No. 192511124), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____